

0073A- Compilation Techniques

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

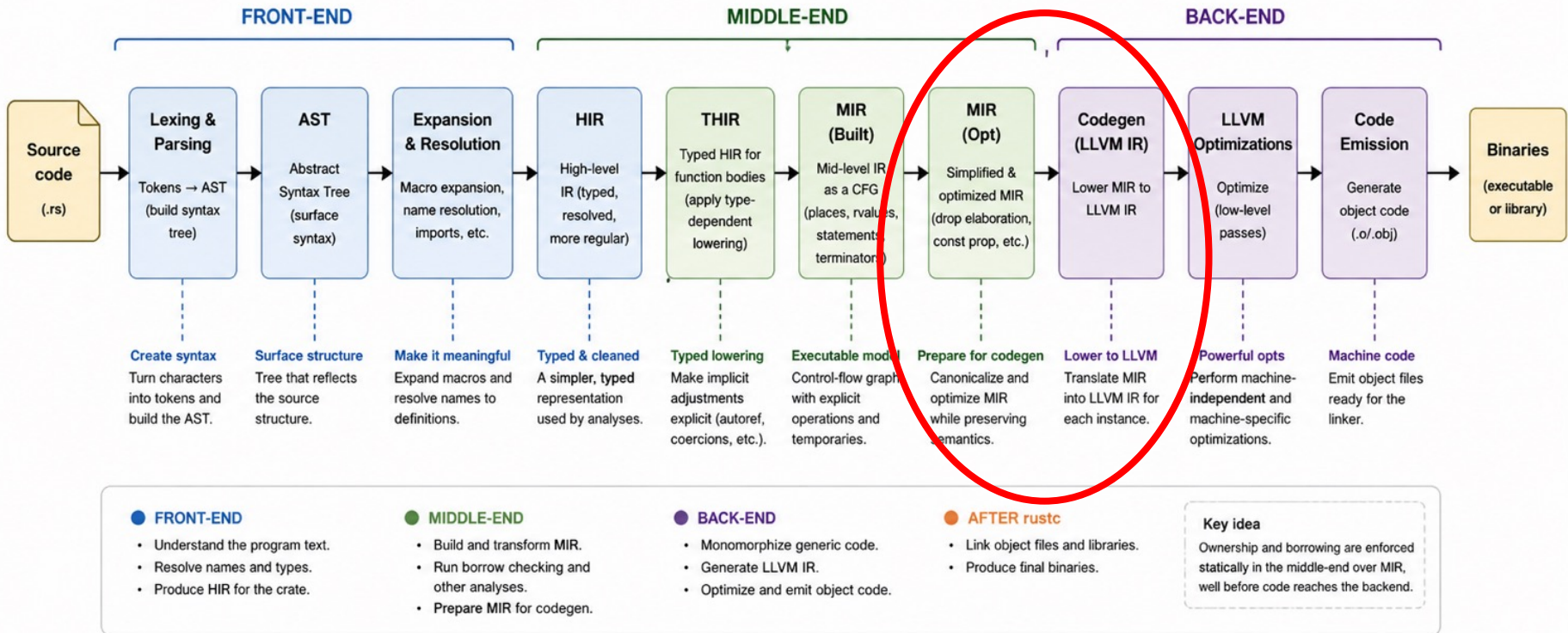
<http://pages.di.unipi.it/corradini/>

CompTech-26: RUST #5

Aspects of Rust Compilation (5)

- Towards code generation
- MIR optimization
- Monomorphization
- Mono Item Collection
- Codegen Units
- Lowering MIR to codegen IR

The rustc Compilation Pipeline



AST = Abstract Syntax Tree • HIR = High-Level IR • THIR = Typed HIR • MIR = Mid-Level IR • CFG = Control-Flow Graph

From MIR to binaries

- After borrow checking on the MIR, the rustc backend runs several steps to produce the output (executable code)
- Several backends supported:
 - LLVM IR (typically)
 - Cranelift IR
 - GCC IR
- LLVM IR is stable, ready for further optimizations
- Cranelift more experimental
- GCC IR is an internal IR of the compiler

Cranelift codegen backend for rust

- Cranelift is a *fast, secure, relatively simple* and *innovative* compiler backend.
- It takes an intermediate representation of a program generated by some frontend and compiles it to executable machine code.
- Cranelift IR is also a compiler IR with *basic blocks, SSA-like values, control flow*, etc., but it is designed for **fast compilation** and simpler backend architecture.
- The Cranelift backend's goal is especially attractive for debug/local builds.
- Its most common use is as a WebAssembly compiler.

Main phases of codegen

- Cleanup after borrow checking, removing from MIR all infos used for analysis and not needed for codegen
- Runtime MIR preparation (including Drop elaboration)
- MIR optimizations, still generic code
- *Monomorphization collection*: identifying all the **concrete types** which are instances of a generic type for which monomorphization has to be performed.
- Lowering of the MIR to a codegen IR **for each concrete type** we collected.
- Codegen backend, which runs a bunch of optimization passes, generates executable code, and links together an executable binary.

Complexity of codegen due to:

- Support for multiple codegen backends. The codegen code is generic over the codegen implementation. Several layers of abstraction.
- Codegen happens asynchronously in another thread for performance.

The actual codegen is done by a third-party library (either of the 3 backends).

MIR optimizations

- **MIR optimizations** improve MIR before backend code generation. Run after **borrow checking**, producing a cleaner and more efficient MIR before lowering to target IR.
- They improve both runtime performance and compilation speed. Better MIR leads to better generated machine code and reduces optimization work required later (e.g. by LLVM).
- Optimizations effective because **MIR is still generic**. MIR optimizations happen *before monomorphization*, so the improved version is reused for all concrete instantiations of that generic function or type.
- MIR optimization implemented as a sequence of passes. Some are mandatory, some only validate invariants or perform checks, some are enabled only in optimized/release builds.
- Typical MIR optimizations simplify and clean the control-flow graph. Examples include **constant propagation**, **dead-code elimination**, **copy propagation**, **CFG simplification**, and **inlining**.
- The **optimized_mir** query drives the process.

Defining optimization passes

The [run_optimization_passes](#) function defines the list of passes to run and the order.

- It contains an array of passes to run.
- Each pass in the array is a struct that implements the [MirPass](#) trait. The array is an array of [&dyn MirPass](#) trait objects. Typically, a pass is implemented in its own module of the [rustc_mir_transform](#) crate.

Some examples of passes are:

- [CleanupPostBorrowck](#): Remove some of the info that is only needed for analyses, rather than codegen.
- [ConstProp](#): Does constant propagation.
- Easily extensible with new optimization passes: see <https://rustc-dev-guide.rust-lang.org/mir/optimizations.html#quickstart-for-adding-a-new-optimization>

Optimization level

```
rustc main.rs -C opt-level=X
```

with X one of [0,1,2,3,s,z]

Available optimization levels:

- 0 → no optimization (fast compilation, good for debugging)
- 1 → basic optimizations
- 2 → standard optimized build
- 3 → aggressive optimizations
- s → optimize for binary size
- z → optimize even more for size

`cargo build` → level 0

`cargo build -release` → level 3

Implementations of generics (universal polymorphism)

Language	Bound mechanism	Main implementation idea	Main tradeoff
Haskell	Type classes	Dictionary passing + parametric polymorphism	Compact code, indirect calls unless optimized
Java	Interfaces / bounds	Type erasure	Compact, less runtime type specificity, indirection
C++	Templates / concepts	Monomorphization	Fast/specialized, code bloat
C#	Interfaces / constraints	Reified generics, shared/specialized hybrid	More runtime type info, efficient value types

Monomorphization in Rust

- Rust has extensive support for generic types, supporting **bounded explicit universal polymorphism**.
- Bounds are expressed using **Traits**.
- Rust *monomorphizes* all generic types: the compiler generates a different copy of the code of a generic function for each concrete type needed.
- Monomorphization is the first step in the backend of the Rust compiler: the generic MIR is instantiated before code generation.
- Same tradeoff as in C++: Fast/specialized, code bloat

Collection

- For each generic thing the compiler must *collect* all concrete types that instantiate it.
- The code performing this collection is called the *monomorphization collector*.
- The monomorphization collector is run just before MIR lowering and codegen.
- [rustc_codegen_ssa::base::codegen_crate](#) calls the [collect and partition mono items](#) query, which does monomorphization collection and then partitions them into [codegen units](#).

Mono item collection

(`rustc_monomorphize` module)

- **Mono item collection determines everything that must generate backend code.**

The collector finds all items that will produce LLVM/backend artifacts: functions, methods, closures, statics, and drop glue. It must also discover every concrete `monomorphized instance of generic code`, including generics imported from other crates.

- **A “mono item” represents one backend artifact.**

A mono item corresponds to something that becomes a function or global object in the generated IR. Mono items depend on each other (e.g., one function calling another), forming a directed `mono item graph`.

- **The collection algorithm works in two phases:**

- Find graph roots by traversing the crate HIR and collecting public/non-generic items.
- Starting from those roots, recursively inspect MIR to discover all used mono items and their concrete type instantiations.

Mono item collection (2)

(`rustc_monomorphize` module)

- **Uses are discovered from MIR, not just explicit calls.**

Edges in the mono item graph arise from:

- function/method calls,
- taking references to functions,
- drop glue generation,
- trait-object unsizing casts (which require vtables),
- and cross-crate generic/inlined functions.

- **Rust supports both lazy and eager collection strategies.**

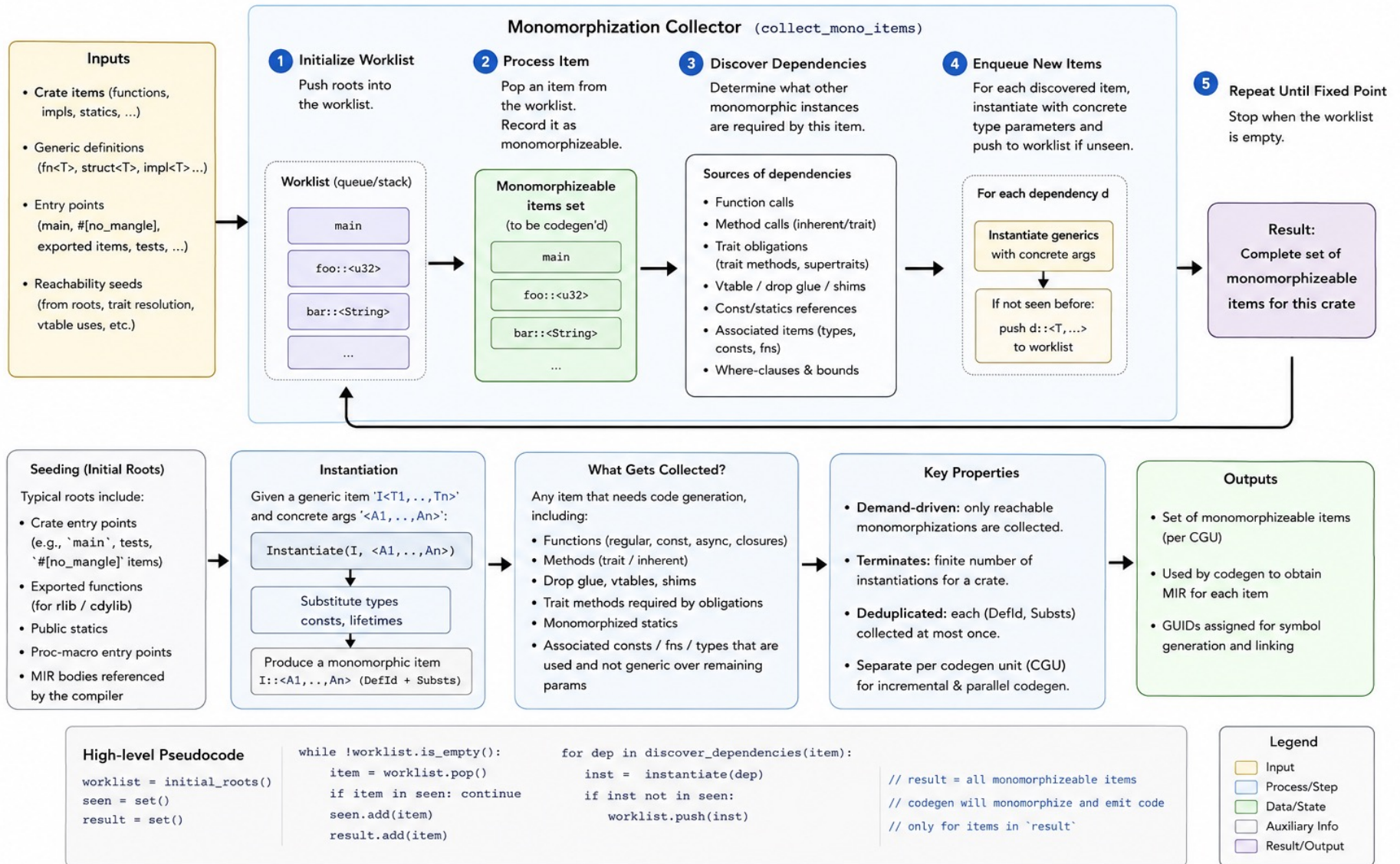
- **Lazy collection:** instantiate only items actually used, minimizing generated code.
- **Eager collection:** instantiate more items proactively (useful for incremental compilation and stable recompilation behavior).

- **The collector also evaluates constants and tracks “mentioned items.”**

To avoid optimization-dependent compilation errors, `rustc` tracks not only items that are actually used after optimization, but also items that syntactically appear in MIR (“mentioned items”). This ensures constant-evaluation failures are reported consistently even if dead code is later optimized away.

```
fn print_val<T: Display>(x: T) {  
    println!("{}", x);  
}  
  
fn call_fn(f: &dyn Fn(i32), x: i32) {  
    f(x);  
}  
  
fn main() {  
    let print_i32 = print_val::<i32>;  
    call_fn(&print_i32, 0);  
}
```

rustc Monomorphization Collector: Logic Overview



Codegen Units (CGUs)

- The collection of mono items generated by the monomorphizier is partitioned into **codegen units (CGUs)** by the **partitioner**.
- A codegen unit is a set of (mono-item, linkage) pairs that becomes one **LLVM module**. The partitioner decides which functions, statics, closures, and monomorphizations go into which CGU.
- Partitioning is crucial for incremental compilation performance.
- LLVM recompiles and optimizes entire modules, not individual functions. Therefore, if a codegen unit changes, the whole LLVM module must be rebuilt and re-optimized from scratch.
- There is a trade-off between incremental recompilation speed and runtime performance.
 - **Many small CGUs** → minimal recompilation after changes, faster incremental builds, slower executables (inter-procedure optimizations and inlining not possible)
 - **Few large CGUs** → better LLVM optimization and inlining, faster executables.

Codegen Units (2)

The partitioner balances these competing goals. `rustc` uses a heuristic based on source-level modules.

For each source module, `rustc` creates:

- one CGU for `stable non-generic code`,
- one CGU for `volatile monomorphized generic instances`.
- This isolates changes caused by generic instantiations and reduces unnecessary recompilation.
- Generic references are especially problematic for incremental compilation.
- Adding or removing a reference to a generic function may create or eliminate monomorphized instances, forcing recompilation even when the generic function body itself did not change.
- Inlining strongly affects partitioning decisions.
- LLVM can inline only when the callee body is available inside the same LLVM module. Therefore the partitioner duplicates eligible functions across CGUs when needed. `rustc` mainly treats functions marked `#[inline]` as candidates for cross-CGU inlining.

Lowering MIR to a Codegen IR

- After mono item collection, rustc lowers **MIR** into a **backend IR**. Usually this target IR is **LLVM IR**, although other backends such as **Cranefift** and **GCC** are also supported.
- Monomorphization happens during this lowering process.
- Code generation starts in **codegen_crate** and reaches **codegen_mir**.
- The lowering logic is divided by MIR constructs. Different modules handle different MIR elements:
 - `block` → basic blocks and terminators (especially function calls and unwinding),
 - `statement` → MIR statements,
 - `operand` → operands,
 - `place` → memory/place references,
 - `rvalue` → computations and temporary values.

Lowering MIR to a Codegen IR (2)

- **Simple analysis passes run before translation.**
rustc performs lightweight analyses to produce cleaner LLVM IR. For example, it detects SSA-like variables so they can be emitted directly in SSA form instead of relying entirely on LLVM optimizations like `mem2reg`.
- **MIR basic blocks usually map directly to LLVM basic blocks.**
Most MIR blocks become one LLVM block, although operations like assertions, intrinsics, or complex calls may expand into multiple LLVM basic blocks.
- LLVM-specific code generation uses **intrinsics** (special built-in LLVM operations) and the **builder interface**.

rustc: Code Generation Phase (after borrowck) → Target IR Generation

Input of this phase: Borrow-checked MIR (MIR after borrowck / region inference / dataflow checks)
Output of this phase: Target IR for one of the supported backends (LLVM IR, Cranelift IR, GCC GIMPLE)

